

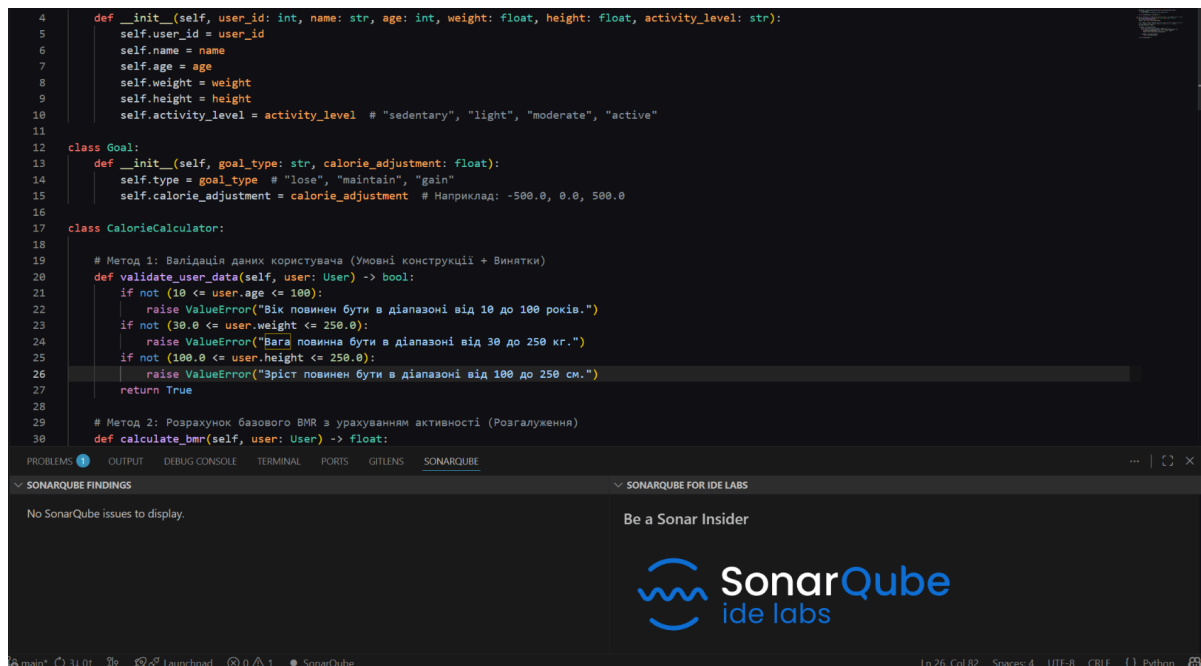
ЛАБОРАТОРНА РОБОТА 4

РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

4.2 Початкова кількість issues та попереджень



```
4 def __init__(self, user_id: int, name: str, age: int, weight: float, height: float, activity_level: str):
5     self.user_id = user_id
6     self.name = name
7     self.age = age
8     self.weight = weight
9     self.height = height
10    self.activity_level = activity_level # "sedentary", "light", "moderate", "active"
11
12 class Goal:
13     def __init__(self, goal_type: str, calorie_adjustment: float):
14         self.type = goal_type # "lose", "maintain", "gain"
15         self.calorie_adjustment = calorie_adjustment # Наприклад: -500.0, 0.0, 500.0
16
17 class CalorieCalculator:
18
19     # Метод 1: Валідація даних користувача (Умовні конструкції + Винятки)
20     def validate_user_data(self, user: User) -> bool:
21         if not (10 <= user.age <= 100):
22             raise ValueError("Вік повинен бути в діапазоні від 10 до 100 років.")
23         if not (30.0 <= user.weight <= 250.0):
24             raise ValueError("Вага повинна бути в діапазоні від 30 до 250 кг.")
25         if not (100.0 <= user.height <= 250.0):
26             raise ValueError("Зріст повинен бути в діапазоні від 100 до 250 см.")
27         return True
28
29     # Метод 2: Розрахунок базового BMR з урахуванням активності (Розгалуження)
30     def calculate_bmr(self, user: User) -> float:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SONARQUBE

SONARQUBE FINDINGS

No SonarQube issues to display.

Be a Sonar Insider

 **SonarQube**
ide labs

Ln 26, Col 82 · Spaces: 4 · UTF-8 · CRLF · Python

```
15     self.calorie_adjustment = calorie_adjustment # Наприклад: -500.0, 0.0, 500.0
16
17     class CalorieCalculator:
18
19         # Метод 1: Валідація даних користувача (Умовні конструкції + Винятки)
20         def validate_user_data(self, user: User) -> bool:
21             if not (10 <= user.age <= 100):
22                 raise ValueError("Вік повинен бути в діапазоні від 10 до 100 років.")
23             if not (30.0 <= user.weight <= 250.0):
24                 raise ValueError("Вага повинна бути в діапазоні від 30 до 250 кг.")
25             if not (100.0 <= user.height <= 250.0):
26                 raise ValueError("Зріст повинен бути в діапазоні від 100 до 250 см.")
27             return True
28
29         # Метод 2: Розрахунок базового BMR з урахуванням активності (Розгалуження)
30         def calculate_bmr(self, user: User) -> float:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SONARQUBE

E902 Не удается найти указанный файл. (os error 2)
--> src:1:1

Found 1 error.
PS C:\Users\Liza\bse-lr1-Terekhova> ruff check src/nutrition.py
E902 Системе не удается найти указанный путь. (os error 3)
--> src\nutrition.py:1:1

Found 1 error.
PS C:\Users\Liza\bse-lr1-Terekhova> ruff check
All checks passed!
PS C:\Users\Liza\bse-lr1-Terekhova>

Ln 26, Col 82 Spaces: 4 UTF-8 CRLF Python Python 3.13

4.3 Рефакторинг

4.3.1 Метод validate_user_data - Magic Numbers (магічні числа)

Було

```
def validate_user_data(self, user: User) -> bool:
```

```
if not (10 <= user.age <= 100):
```

```
    raise ValueError("Вік повинен бути в діапазоні від 10 до 100 років.")
```

```
if not (30.0 <= user.weight <= 250.0):
```

```
    raise ValueError("Вага повинна бути в діапазоні від 30 до 250 кг.")
```

```
if not (100.0 <= user.height <= 250.0):
```

```
    raise ValueError("Зріст повинен бути в діапазоні від 100 до 250 см.")
```

```
return True
```

Стало

```
MIN_AGE = 10
```

```
MAX_AGE = 100
```

```
MIN_WEIGHT = 30.0
```

```
MAX_WEIGHT = 250.0
```

```
MIN_HEIGHT = 100.0
```

```
MAX_HEIGHT = 250.0
```

```

def validate_user_data(self, user: User) -> bool:
    if not (self.MIN_AGE <= user.age <= self.MAX_AGE):
        raise ValueError("Вік поза допустимим діапазоном.")

    if not (self.MIN_WEIGHT <= user.weight <= self.MAX_WEIGHT):
        raise ValueError("Вага поза допустимим діапазоном.")

    if not (self.MIN_HEIGHT <= user.height <= self.MAX_HEIGHT):
        raise ValueError("Зріст поза допустимим діапазоном.")

    return True

```

У коді використовуються магічні числа (10, 100, 30.0, 250.0), які не пояснюють своє значення. Це ускладнює зміну правил валідації та зменшує читабельність. Винесення в константи робить код зрозумілим і легким для підтримки.

4.3.2 Метод calculate_bmr() - Magic Numbers + SRP (частково)

Було

```

def calculate_bmr(self, user: User) -> float:
    self.validate_user_data(user)

    base_bmr = (10 * user.weight) + (6.25 * user.height) - (5 * user.age) + 5

    activity_multipliers = {
        "sedentary": 1.2,
        "light": 1.375,
        "moderate": 1.55,
        "active": 1.725
    }

    multiplier = activity_multipliers.get(user.activity_level.lower())

    if not multiplier:
        raise ValueError("Невідомий рівень активності.")

    return round(base_bmr * multiplier, 2)

```

Стало

```
ACTIVITY_MULTIPLIERS = {  
    "sedentary": 1.2,  
    "light": 1.375,  
    "moderate": 1.55,  
    "active": 1.725  
}
```

```
def calculate_bmr(self, user: User) -> float:  
    self.validate_user_data(user)
```

```
    base_bmr = (  
        (10 * user.weight)  
        + (6.25 * user.height)  
        - (5 * user.age)  
        + 5  
    )
```

```
    multiplier = self.ACTIVITY_MULTIPLIERS.get(user.activity_level.lower())
```

```
    if multiplier is None:  
        raise ValueError("Невідомий рівень активності.")
```

```
    return round(base_bmr * multiplier, 2)
```

Магічні числа коефіцієнтів активності були винесені в константу, щоб зробити їх легко змінюваними. Також покращена перевірка None замість falsy-значення для точності. Це підвищує читабельність і зменшує ризик помилок при зміні формули.

4.3.3 calculate_target_calories() — SRP + Long Method + Complexity

Було

```
def calculate_target_calories(self, user: User, goal: Goal) -> float:  
    bmr = self.calculate_bmr(user)  
    target = bmr + goal.calorie_adjustment
```

```
    min_safe_calories = 1200.0
```

```
if target < min_safe_calories:
while target < min_safe_calories and goal.calorie_adjustment < 0:
goal.calorie_adjustment += 50
target = bmr + goal.calorie_adjustment
```

```
if target < min_safe_calories:
return min_safe_calories
```

```
return round(target, 2)
```

Стало

```
MIN_SAFE_CALORIES = 1200.0
```

```
CALORIE_ADJUSTMENT_STEP = 50
```

```
def ensure_safe_calories(self, bmr: float, calorie_adjustment: float) -> float:
target = bmr + calorie_adjustment
```

```
while target < self.MIN_SAFE_CALORIES and calorie_adjustment < 0:
calorie_adjustment += self.CALORIE_ADJUSTMENT_STEP
target = bmr + calorie_adjustment
```

```
return max(target, self.MIN_SAFE_CALORIES)
```

```
def calculate_target_calories(self, user: User, goal: Goal) -> float:
bmr = self.calculate_bmr(user)
```

```
safe_target = self.ensure_safe_calories(bmr, goal.calorie_adjustment)
```

```
return round(safe_target, 2)
```

Метод виконував одразу кілька завдань: розрахунок калорій, перевірку безпечності та коригування значень. Це порушувало принцип єдиної відповідальності (SRP) та збільшувало складність коду. Логіку перевірки безпечності було винесено в окремий метод, що зменшило рівень вкладеності та покращило читабельність. Тепер кожен метод відповідає лише за одне завдання.

4.4 Звіт рефакторингу

4.4.1 Звіт на основі ручного тестування

```
.....
-----
Ran 10 tests in 0.004s

OK
```

Для методу `validate_user_data`

```
.....
-----
Ran 10 tests in 0.002s

OK
```

Для методу `calculate_bmr()`

```
.....
-----
Ran 10 tests in 0.002s

OK
PS C:\Users\Liza\bse-lr1-Terekhova>
```

Для методу `calculate_target_calories()`

4.4.2 Звіт на основі pytest

```
PS C:\Users\Liza\bse-lr1-Terekhova> pytest --cov=nutrition --cov-report=term
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\Liza\bse-lr1-Terekhova
plugins: cov-7.1.0
collected 10 items

test_nutrition.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.0-final-0

Name      Stmts  Miss  Cover
-----
nutrition.py  47      1   98%
TOTAL      47      1   98%

===== 10 passed in 0.22s =====
PS C:\Users\Liza\bse-lr1-Terekhova>
```

4.5 Порівняння метрик до та після

4.5.1 До

SonarLint issues: у вихідній версії коду помилки не були виявлені.

Ruff: попередження відсутні.

Цикломатична складність: код мав низьку цикломатичну складність, оскільки містив прості умовні конструкції.

4.5.2 Після

SonarLint issues: після рефакторингу також не було виявлено жодних проблем у SonarLint.

Ruff: попередження відсутні.

Цикломатична складність: після рефакторингу складність не змінилась і залишається низькою.

4.6 Підсумкова рефлексія

Під час виконання Code Review я зрозуміла важливість уважного аналізу коду та виявлення потенційних проблем ще до етапу рефакторингу. Аналіз чужого коду допоміг краще помічати code smells і типові помилки проєктування.

Також я побачила, що рефакторинг потрібно виконувати поступово, постійно перевіряючи тести, щоб уникати регресій.

Окремо я звернула увагу на важливість дотримання домовленого процесу роботи в команді, оскільки це впливає на ефективність взаємної перевірки.